CloudCost AI: An Intelligent FinOps and Cloud Cost Optimization Platform for AWS Cost Monitoring, Anomaly Analysis, and Recommendations

Prof. Sarin Deore
Department of Information Technology
A. C. Patil College of Engineering
Kharghar, Navi Mumbai, India

Pramod Shinde
Department of Information Technology
A. C. Patil College of Engineering
Kharghar, Navi Mumbai, India
pramodsshine@acpce.ac.in

Aditya Jadhav
Department of Information Technology
A. C. Patil College of Engineering
Kharghar, Navi Mumbai, India
adityajadhav@acpce.ac.in

Mayuresh Jadhav
Department of Information Technology
A. C. Patil College of Engineering
Kharghar, Navi Mumbai, India
mayureshjadhav@acpce.ac.in

Abstract*—Cloud computing gives organizations on-demand access to computing, storage, and database resources, but the same elasticity makes expenditure difficult to predict and control. Unused or oversized resources and unexpected increases in monthly bills are common causes of avoidable cost, while many monitoring approaches concentrate on displaying billing data and leave trend interpretation, anomaly identification, and optimization to the user. This paper presents CloudCost AI, a FinOps and cloud cost optimization platform that unifies these functions for Amazon Web Services (AWS). A Spring Boot (Java) backend validates AWS credentials with AWS Security Token Service, imports billing data (AWS Cost and Usage Report 2.0 exported to Amazon S3, and live AWS Cost Explorer queries), discovers resources, and stores everything in PostgreSQL to preserve cost history. Deterministic analytical modules provide service- and region-level cost analysis, top-cost-driver insights, threshold-based cost anomaly flags, rule-based optimization recommendations, budget tracking with in-app and e-mail alerts, and PDF reports, presented in a Next.js dashboard. Access uses Spring Security, application sessions with BCrypt-hashed passwords, and a separate AWS connection token, and AWS credentials are held only in server memory. The completed system was evaluated on one AWS account for the billing month of August 2026. It reported a total cost of USD 7.24, identified AWS Cost Explorer (53.58%) and Amazon RDS (27.07%) as the leading cost drivers, flagged four cost anomalies, and generated five recommendations. Because the account is small and has one month of history, no accuracy claims are made, forecasting is not implemented, and comparison with existing tools is qualitative. Limitations and directions for machine-learning-based forecasting and multi-cloud support are discussed.

***Index Terms*—Cloud computing, FinOps, AWS, cloud cost optimization, anomaly detection, budget monitoring, Spring Boot, cloud resource management, PostgreSQL.**

I. INTRODUCTION

Cloud computing has become a standard delivery model for computing, storage, networking, and database services. The National Institute of Standards and Technology characterizes it as a model for convenient, on-demand network access to a shared pool of configurable resources that can be rapidly provisioned and released [1]. Its economic appeal, namely elastic capacity and pay-as-you-go pricing, was identified early as a defining property of the model [2]. The same properties, however, shift financial control from a planned capital budget to a continuous stream of usage-based charges that engineers can change with a few API calls.

Managing this expenditure is now a discipline in its own right. FinOps, a portmanteau of finance and operations, is an operating practice in which engineering, finance, and business teams share accountability for cloud spend [3]. In practice, avoidable cost arises from resources that are provisioned but idle, instances that are larger than the workload requires, inefficient allocation across services and regions, and unexpected increases that only become visible when the monthly invoice arrives. Guidance from the FinOps Foundation notes that a cost anomaly is a level of spending that differs from what is normal or expected, and that timely detection is what allows a team to react before the impact accumulates [4].

Monitoring these costs manually is difficult. Billing data is voluminous, is organized by service, account, region, and usage type, and changes daily. Reading a billing console to decide whether a rise is legitimate growth or waste requires the analyst to compare periods, recall earlier baselines, and correlate spend with the resources that caused it. Cloud providers expose cost-management tools and programmatic interfaces for this purpose; for AWS these include the Cost Explorer API for querying cost and usage and retrieving forecasts [5]. Conventional tooling, however, tends to emphasize the display of billing information. Historical analysis is often limited to the view the console offers, trend interpretation is left to the user, unusual increases are not surfaced together with the context needed to act, and optimization advice is fragmented across separate tools.

Recent research has addressed parts of this problem. ABACUS automates budget analysis and usage surveillance for cloud spend [6]. Machine learning has been applied to long-term prediction of cloud resource usage for FinOps-driven resource planning [7], [8], and an autonomous, LLM-based FinOps agent has been proposed to consolidate

heterogeneous cost data and generate optimization recommendations [9]. These works are valuable, but they emphasize individual capabilities (budgeting, prediction, or agent-based analysis) or rely on specific commercial platforms, and none is described as a compact, self-hostable application that ties AWS data collection, persistent cost history, analytics, and secured dashboard access together for small teams. This gap is the research gap addressed here: a lack of a simple, unified interface that combines cost monitoring, historical analysis, anomaly identification, budget alerts, and actionable recommendations over data drawn directly from an AWS environment.

This motivates CloudCost AI, a platform whose main objective is to connect with AWS, collect cloud resource and billing information, store it as historical data, analyze expenditure and resource usage, and present meaningful insights through a user-friendly dashboard. Its target users are cloud administrators, DevOps engineers, developers, students and researchers, and small organizations managing AWS resources.

The contributions of this paper are as follows. (1) A unified framework for AWS cloud cost monitoring and analysis is presented and implemented. (2) Cloud billing and resource data acquired through AWS APIs are integrated with a centralized PostgreSQL database that retains historical cost data and analytical results. (3) A dashboard visualizes cost information for end users. (4) The platform provides analytical capabilities for service- and region-level cost analysis, top-cost-driver insights, anomaly identification, rule-based optimization recommendations, budget tracking with in-app and e-mail alerts, and PDF reporting on the stored data. (5) Access is protected through Spring Security, application sessions with BCrypt-hashed passwords, and a separate short-lived AWS connection token, and AWS credentials are kept only in server memory. (6) A practical reference architecture combining AWS, Spring Boot, PostgreSQL, and a web frontend is documented, together with a functional evaluation on real AWS data and an explicit discussion of limitations. All contributions listed correspond to functionality implemented in the completed project.

The remainder of the paper is organized as follows. Section II gives background. Section III reviews related work. Sections IV and V describe the methodology and architecture, and Section VI the implementation. Sections VII to IX present the dataset, results, and ablation status. Sections X to XII discuss security, privacy, scalability, limitations, future scope, and conclusions.

II. BACKGROUND AND PROBLEM DOMAIN

A. Cloud Computing and AWS

Amazon Web Services provides compute, storage, database, networking, and analytics services that are billed by usage. Each resource, for example a virtual machine instance, a storage volume, or a managed database, accrues charges according to its type, size, region, and running time. Billing information is aggregated by service, linked account, region, and usage type, and AWS exposes it programmatically through cost-management APIs that return cost and usage metrics for a chosen period and grouping and can return forecasts based on past costs [5]. Effective cost management therefore requires both billing data (what was charged) and resource data (what was provisioned and how heavily it was used).

B. FinOps

FinOps brings financial accountability to the variable-spend model of the cloud [3]. It is commonly described as an iterative cycle of three phases: Inform, which provides visibility and allocation of cost; Optimize, which improves rates and usage; and Operate, which embeds continuous improvement [9]. CloudCost AI primarily supports the Inform phase through cost visibility and historical analysis, and supports the Optimize phase through anomaly identification and recommendations.

C. Cloud Cost Optimization

Cloud cost optimization comprises several practices. *Rightsizing* matches resource capacity to observed demand, so an instance that consistently uses a small fraction of its capacity becomes a candidate for a smaller type. *Resource utilization* analysis measures how much of the provisioned capacity is actually used. *Unused resources*, such as detached volumes or stopped but still billed assets, represent spend without value. *Cost monitoring* tracks expenditure continuously, *cost allocation* attributes it to services, accounts, or teams, and *historical analysis* supplies the baseline against which present spending is judged. The total cost over a period is the sum of the individual costs:

$$C_{total} = \sum_{i=1}^{n} C_i \tag{1}$$

where C_i is the cost of resource or service i and n is the number of considered resources or services.

D. Cloud Cost Anomalies

A cost anomaly is spending that departs from the normal or expected level [4]. Typical causes include a misconfigured or forgotten resource, an unexpected surge in usage, data transfer charges, or a change in resource type. Anomaly detection in general identifies observations that deviate from normal behavior in a dataset [10]. For cost data, the change between consecutive periods and its relative magnitude are basic indicators:

$$\Delta C_t = C_t - C_{t-1} \tag{2}$$

$$P_t = (C_t - C_{t-1}) / C_{t-1} \times 100 \tag{3}$$

where C_t and C_{t-1} are the costs of the current and previous periods, ΔC_t is the absolute variation, and P_t is the percentage change. CloudCost AI also flags individual days whose cost is far above the average of the earlier days in the selected month. For a daily series $C_1, \ldots, C_n$ with $n \geq 8$, the baseline is the mean of all days except the last,

$$\bar{C} = (1/(n-1)) \sum_{j=1}^{n-1} C_j \tag{4}$$

and day t is flagged when

$$A_t = (C_t - \bar{C}) / \bar{C} \times 100 \geq \tau, \ \tau = 50 \tag{5}$$

so a day is flagged when its cost is at least 50% above the baseline. This is a simple statistical rule, not a trained model.

E. Budget Utilization

Budget tracking compares accumulated spend S with a monthly budget B:

$$U = (S / B) \times 100 \quad (6)$$

The platform labels the budget SAFE below 50%, NORMAL from 50%, WARNING from 80%, CRITICAL from 90%, and EXCEEDED from 100%, and raises alerts when configured thresholds (by default 50%, 80%, 90%, and 100%) are reached.

III. RELATED WORK

This section reviews verifiable prior work on FinOps, cloud cost optimization, forecasting, and anomaly detection, and states how CloudCost AI differs. Descriptions are based on the cited publications; where a publication's detail could not be verified, the statement is kept general.

Budgeting and surveillance. Deochake proposes ABACUS (Automated Budget Analysis and Cloud Usage Surveillance), a FinOps service that applies FinOps principles to cost attribution, budgeting, and alerting, including enforcing budgets and alerting teams when spending breaches a threshold [6]. Its contribution is automation of budget governance. Its emphasis is on budgets and alerts rather than on an integrated view combining historical analysis, forecasting, and recommendations. CloudCost AI instead stores cost history and provides these analyses in one dashboard.

Machine-learning-based resource usage prediction. Nawrocki and Smendowski present a Cloud Resource Usage Optimization System that uses machine learning to predict long-term resource usage and builds resource reservation plans across virtual machine types for high-performance computing on Google Cloud Platform [7]. A later study proposes a multi-time-series forecasting approach with a hybrid ensemble for cloud resource utilization [8], and the same authors survey cloud resource consumption optimization methods [11]. These works contribute forecasting techniques and rigorous model design, but they are algorithm-centered, target HPC workloads or general utilization series, and do not describe an end-user, authenticated platform over live AWS billing data. CloudCost AI is complementary: it provides the application layer (data acquisition, persistence, security, and visualization) into which such forecasting models could later be integrated.

Agent-based FinOps. Vo et al. build a multi-agent FinOps agent that uses a unified GraphQL schema over IBM Turbonomic and Apptio data and a natural-language-to-GraphQL layer to retrieve data and generate optimization recommendations, evaluated with several language models on mock datasets [9]. The approach addresses heterogeneous data sources and conversational access. Limitations relevant to this work are its dependence on commercial platforms and language models and its evaluation on mock datasets. CloudCost AI is narrower but simpler: it targets AWS directly, runs on a conventional Spring Boot and PostgreSQL stack, and does not depend on a language model.

Anomaly detection. Islam et al. introduce a large industrial dataset of IBM Cloud telemetry and evaluate machine-learning models for anomaly detection [12], and Chandola et al. survey anomaly detection techniques in general [10]. These works address operational telemetry rather than billing data, but they inform the design space for detectors. The FinOps Foundation describes cost anomaly management, including detection based on expected ranges or thresholds and the value of alerting promptly [4]. CloudCost AI applies anomaly identification to cost series within the same platform that stores them.

Provider-native tools. AWS Cost Explorer and AWS Cost Anomaly Detection provide cost visualization, forecasts, and machine-learning-based anomaly alerts [5], [9]. They are mature and are not claimed to be outperformed by CloudCost AI. Their scope is the provider's own console and services; CloudCost AI's contribution is an independently deployable, unified interface for learning, research, and small-team use whose data, analysis logic, and security controls are under the operator's control. Table I summarizes the comparison. Because no head-to-head experiment against these systems was performed, the comparison is qualitative.

TABLE I
QUALITATIVE COMPARISON OF CLOUDCOST AI WITH RELATED APPROACHES

Existing Approach	Main Technique	Advantages	Limitations	Relation to CloudCost AI
ABACUS [6]	Automated budget analysis, cost attribution, alerting	Budget governance and alerts across teams	Emphasis on budgets and alerts; integrated forecasting and recommendations not its focus	CloudCost AI adds stored history, anomaly, insight, and recommendation views in one dashboard
CRUOS, Nawrocki and Smendowski [7]	ML-based long-term resource usage prediction for HPC on GCP	Rigorous prediction; reservation planning	HPC and GCP focus; no end-user cost platform described	CloudCost AI supplies the application layer; models could be integrated later
Multi-time-series forecasting, Smendowski and Nawrocki [8]	Hybrid ensemble forecasting of cloud utilization	Improved forecasting design for utilization series	Algorithm-centered; no billing dashboard or authentication described	Candidate forecasting component for future work
FinOps agent, Vo et al. [9]	LLM multi-agent system with unified GraphQL schema	Handles heterogeneous sources; natural-language access	Relies on commercial platforms and LLMs; evaluated on mock data	CloudCost AI is AWS-direct, LLM-free, and evaluated on AWS data
Cloud anomaly detection, Islam et al. [12]	ML anomaly detection on IBM Cloud telemetry	Large real-world dataset; ML benchmarks	Operational telemetry, not cost data	Informs detector design; CloudCost AI targets cost series
AWS Cost Explorer and Cost Anomaly Detection [5], [9]	Provider-native visualization, forecast, and ML anomaly alerts	Mature, integrated with AWS billing	Provider-specific console; limited user control over logic	CloudCost AI is an independent, self-hosted unified interface (qualitative comparison only)

In summary, the reviewed literature offers strong individual components, namely budgeting, forecasting, agent-based analysis, and anomaly detection, but few descriptions of a compact and secured application that integrates AWS data acquisition, a persistent historical store, several analytical functions, and a dashboard. CloudCost AI is positioned to fill this practical gap. No claim of algorithmic novelty is made in this paper.

IV. PROPOSED METHODOLOGY

CloudCost AI processes data through thirteen stages, from connecting an AWS account to presenting results. Details that are still to be confirmed by the authors are highlighted.

1) AWS account/resource connection. The user registers and logs in; the backend then issues a 24-hour application session token. To connect AWS, the user supplies an access key, secret key, and region, which the backend validates with the AWS Security Token Service (STS) GetCallerIdentity call. The credentials are held only in backend memory and are not written to PostgreSQL; the resulting AWS connection token, which the data APIs require, expires after 30 minutes.

2) Cloud billing and resource-data collection. Billing data come from the AWS Cost and Usage Report (CUR) 2.0 delivered to Amazon S3, which the backend reads and imports on a manual synchronization request, and from live AWS Cost Explorer queries made on demand. Resources are obtained by discovery calls.

3) Data acquisition through AWS APIs/services. The AWS SDK for Java (v2) is used. STS verifies identity; EC2, S3, RDS, and Lambda clients discover resources (instances, volumes, VPCs, security groups, buckets, databases, functions); the S3 client reads the CUR files; and Cost Explorer GetCostAndUsage returns unblended cost at daily granularity grouped by region and service [5]. Because Cost Explorer requests are themselves billable, they are made only on demand.

4) Data validation. For CUR files the importer requires a configured bucket and at least one supported CSV or gzip file, requires the usage-start-date and unblended-cost columns, decompresses gzip, checks that each row belongs to the connected account, ignores invalid dates and cost values, falls back to the start date when the end date is missing, and skips duplicates using a SHA-256 record key. For Cost Explorer, missing results yield a NO_DATA status, and invalid month parameters are rejected.

5) Data preprocessing. Records are normalized to billing period, service, region, usage, cost, currency, usage type, resource identifier, and source, and are grouped by their effective billing date.

6) Data storage in PostgreSQL. Billing records, per-account summaries, budgets and alerts, recommendations, notification preferences and logs, users, and sessions are persisted, so that history accumulates and analysis does not require re-querying AWS.

7) Historical cost analysis. Stored costs are aggregated by service and region for a selected month, and total cost follows (1).

8) Cost trend analysis. Daily cost series are produced for the selected month, and the previous-period difference and percentage change follow (2) and (3) when a previous period exists; otherwise the platform reports the value as unavailable. The overview page shows a six-month monthly trend, with chart ranges of 7 days, 30 days, 3, 6, and 12 months. Weekly aggregation is not implemented.

9) Anomaly detection. For a daily series of at least eight points, each day is compared with the mean of all days except the last, and a day at least 50% above that mean is flagged, following (4) and (5).

10) Insights. Services are ranked by cost, the top five are kept, and a top-cost-driver insight is generated when a service costs at least USD 1 or has at least 25% of spend. Further insight rules cover account and service cost increases and unused resources (Table II).

11) Recommendation generation. Four rule families produce recommendations with evidence, a suggested action, a severity, and a status (Table II). Potential savings are reported only where a rule can support them and are shown as not available otherwise.

12) Budgets, alerts, and notifications. The user creates a monthly budget with thresholds (default 50%, 80%, 90%, and 100%); alerts are generated once per budget, threshold, and month. A background worker runs every 15 minutes (first run after one minute) and sends in-app notifications and e-mail through SMTP according to the user's preferences.

13) Dashboard, reporting, and interaction. Results appear in the dashboard, and the user can filter by month, service, and region and export a PDF FinOps report.

TABLE II
IMPLEMENTED ANALYTICAL RULES

Module	Condition	Output
Anomaly	$n \geq 8$ daily points; cost ≥ 50% above mean of all but last day	Percentage above baseline; no severity
Insight: top driver	Top 5 services; cost ≥ USD 1 or share ≥ 25%	High if share ≥ 50%, else Medium; impact = 10% of cost
Insight: account increase	Current > 1.25 × previous	High; impact = increase
Insight: service spike	Current > 1.5 × previous and increase ≥ USD 1	High; impact = increase
Insight: unused resource	Usage = 0 and cost > 0	Medium; impact = resource cost
Rec.: expensive resource	Resource cost ≥ USD 1; top 5	High if ≥ USD 10, else Medium; savings n/a
Rec.: cost increase	Service current > 1.5 × previous	High; savings n/a
Rec.: unused resource	Usage = 0 and cost > 0; top 5	Medium; savings = cost
Rec.: high service cost	S3, RDS, Lambda, or EC2; top 4	High if ≥ USD 10, else Medium; savings n/a
Budget status	Utilization U in (6)	SAFE < 50; NORMAL ≥ 50; WARNING ≥ 80; CRITICAL ≥ 90; EXCEEDED ≥ 100

Algorithm 1 summarizes the processing performed by the backend.

```
Algorithm 1: CloudCost AI processing
Input: user login, AWS keys, month M
Output: cost views, anomalies, insights,
        recommendations, budgets, report
1  if not Authenticate(user): reject
2  session <- CreateSession(user)   // 24 h
3  if not STS.GetCallerIdentity(keys): error
4  token <- ConnectionToken(keys)   // 30 min
5  data APIs require token, else 401
6  billing <- ImportCUR(S3)   // manual sync
   live <- CostExplorer(M)    // on demand
7  inv <- DiscoverResources()
8  if not Validate(billing,live,inv): error
9  Store(Preprocess(billing,live,inv))
10 costs <- Aggregate(M)  // (1),(2),(3)
11 A <- Anomalies(costs)  // (4),(5): n>=8,
                          // >= 50% above mean
12 I <- Insights(costs)   // Table II
13 R <- Recommend(data,costs)  // Table II
14 B <- BudgetStatus(costs)    // (6)
15 Store(R, alerts) into PostgreSQL
16 worker every 15 min: notify (in-app, email)
17 return {costs, A, I, R, B} to dashboard
```

V. SYSTEM ARCHITECTURE AND PROPOSED FRAMEWORK

CloudCost AI follows a layered client-server architecture. The user-facing path is: User → Next.js Dashboard → Spring Boot REST API → Security Layer → Services → PostgreSQL. The AWS path is: AWS (STS, Cost Explorer, CUR 2.0 in S3, resource APIs) → Spring Boot services → PostgreSQL → Analytics → Dashboard. Fig. 1 shows the architecture.

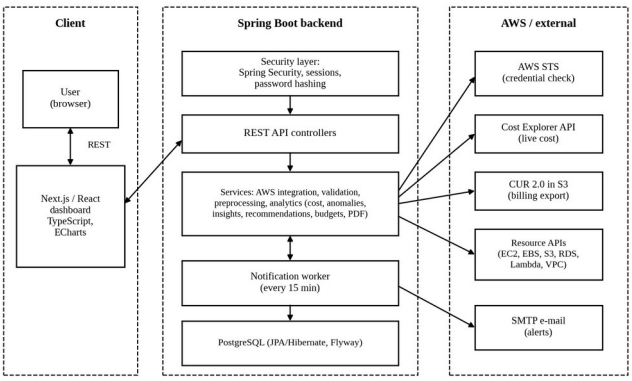


Fig. 1. Overall architecture of the proposed CloudCost AI framework.

Description of Fig. 1. The user works in a browser with the Next.js dashboard, which calls the REST endpoints of the Spring Boot backend. A Spring Security filter checks the AWS connection token of protected data requests before they reach the controllers. Controllers delegate to services that integrate with AWS, validate and preprocess data, and run the analytics. Services read and write PostgreSQL through JPA/Hibernate, with Flyway managing schema migrations. A notification worker periodically evaluates alert preferences and sends e-mail through SMTP. On the AWS side, STS validates credentials, the Cost Explorer API supplies live cost, CUR 2.0 data arrive through S3, and resource APIs supply the inventory.

Frontend dashboard. Next.js, React, and TypeScript, with ECharts for charts.

REST API and security layer. Controllers expose authentication, AWS connection, billing, analytics, budget, notification, and report endpoints; a Spring Security filter protects the data endpoints.

AWS integration. Wraps AWS SDK calls and converts responses to internal models.

Analytics module. Cost analytics, anomaly detection, insights, recommendations, and budgets.

Notification worker. Evaluates preferences every 15 minutes and delivers in-app and e-mail notifications.

PostgreSQL. Stores users, sessions, AWS accounts, billing data and summaries, recommendations, budgets, alerts, and notification records.

Data flow. (i) The user logs in and receives a 24-hour session token. (ii) The user connects an AWS account, which the backend validates with STS, receiving a 30-minute connection token that later data requests must carry. (iii) The backend imports CUR 2.0 data from S3, discovers resources, and fetches live Cost Explorer data on request. (iv) Validated, preprocessed data are stored in PostgreSQL. (v) The analytics read stored data and produce insights, recommendations, and alerts. (vi) The dashboard requests results through the REST API. (vii) The user reviews results, exports reports, and receives notifications. Fig. 2 shows the workflow.

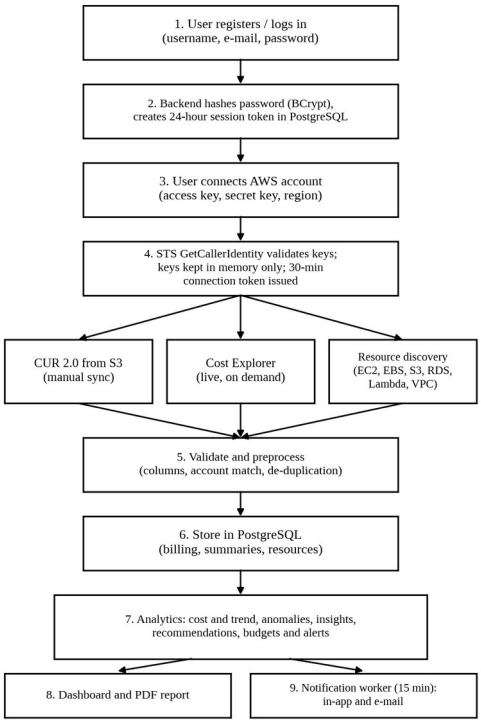


Fig. 2. Activity workflow of the proposed CloudCost AI methodology.

VI. IMPLEMENTATION DETAILS

A. Backend

The backend uses Java 25 and Spring Boot 4.1.0, built with Maven 3.9.16 (Maven wrapper). It uses Spring Web MVC, Spring Data JPA (Hibernate), Spring Security, Spring Validation, Spring Mail, Spring Actuator, and Flyway for database migrations. AWS access uses the AWS SDK for Java v2 (version 2.20.100; STS, EC2, S3, RDS, Lambda, and Cost Explorer clients; a Budgets client is instantiated but no Budgets API call is made). CUR files are parsed with Apache Commons CSV 1.12.0 and PDF reports are generated with OpenPDF 2.0.3. The structure is controller–service–repository, with JSON over REST. API testing used Postman.

B. Database

PostgreSQL is the persistent store (the server version shown in the database client was 18; JDBC driver 42.7.12). Migrations define the tables app_users (with a password hash), app_sessions (token, user, expiry), aws_accounts, aws_billing, aws_billing_summary, aws_recommendations, budgets, budget_alerts, notification_preferences, and notification_logs. The billing table stores account, billing period, service, region, usage, cost, currency, usage type, resource, line-item type, usage dates, a record key for de-duplication, and the data source. AWS access keys are not stored: the migration history removed the original access-key and secret-key columns, and credentials are kept only in memory.

C. Frontend Dashboard

The dashboard uses Next.js 16.2.12, React 19.2.4, and TypeScript 5, with ECharts 6.1 (echarts-for-react) for charts, Axios and TanStack Query for data access, Zustand for state, React Hook Form with Zod for forms, and Tailwind CSS with Radix UI components. Its pages are Overview, Cost Analytics, Resources, AI Insights, Recommendations, Budgets & Alerts, Cloud Accounts, Reports, Settings, and Help (Figs. 3 to 7). A Forecast page is disabled in the navigation.

D. AWS Integration

Table III lists the AWS calls used. Only these interfaces are claimed.

E. Authentication and Security

Application login. The backend provides register, login, current-user, and logout endpoints. Registration requires a username of 3 to 100 characters, a valid e-mail address, and a password of 8 to 128 characters. On success the backend creates an opaque session token (a random UUID with hyphens removed, 32 characters), stores it with its user and an expiry of 24 hours in the app_sessions table, and returns it. Logout deletes the session. The project does not use JWT. The frontend keeps the token in browser local storage when "remember me" is chosen and in session storage otherwise, and sends it as an Authorization bearer header.

Password hashing. Passwords are hashed with Spring Security's BCryptPasswordEncoder in its default configuration, so a random salt is generated per password and only the hash is stored. BCrypt is an adaptive scheme whose cost factor can be raised as hardware improves [13]; the project sets no explicit cost factor.

AWS connection token and request filtering. A custom Spring Security filter requires an X-AWS-Connection-Token header on protected API requests and rejects missing, invalid, or expired tokens with HTTP 401. AWS credentials are validated with STS, held only in server memory, and discarded when the 30-minute connection token expires. CORS allows the configured frontend origin (default localhost:3000), and CSRF protection is disabled because requests are stateless and token-based.

F. Error Handling

Errors are handled endpoint by endpoint; the project has no global exception handler. A missing or expired AWS connection token returns HTTP 401 ("AWS connection is required" or "token is invalid or expired"); invalid login credentials return 401, and an expired application session returns 401. Invalid AWS credentials raise an AwsConnectionException after the STS call fails. Cost Explorer failures and PostgreSQL storage failures are wrapped in the same exception type, and a failed CUR import raises an exception stating that the data could not be imported from S3. Empty Cost Explorer results return status NO_DATA with HTTP 200, a PDF request without data returns HTTP 422, and a failure to load billing data for insights returns HTTP 500 with a retry message. Invalid month parameters raise an IllegalArgumentException asking for the YYYY-MM format, and request validation (for example password length and budget amount) uses Jakarta validation. The dashboard shows "Unavailable" when only one month exists, "No resources found" for empty inventories, "Not available" for unsupported savings, and "No budget configured" when no budget exists. These AWS connection and import exceptions are not mapped by a global handler.

G. Technical Reproducibility

Table IV summarizes the reproducibility information; unavailable values are highlighted.

TABLE III
AWS CALLS USED

Service	Operation (purpose)
STS	GetCallerIdentity (credential validation)
EC2	Describe instances, volumes, VPCs, security groups (inventory)
S3	List buckets, bucket location (inventory); list and get objects (CUR import)
RDS	Describe DB instances (inventory)
Lambda	List functions (inventory)
Cost Explorer	GetCostAndUsage: unblended cost, daily, grouped by region and service

TABLE IV
REPRODUCIBILITY SPECIFICATION

Item	Value
Backend	Java 25; Spring Boot 4.1.0; Maven 3.9.16
Data access	Spring Data JPA/Hibernate; Flyway; PostgreSQL JDBC 42.7.12

Item	Value
Database	PostgreSQL (client showed version 18)
AWS SDK	AWS SDK for Java v2, 2.20.100
Frontend	Next.js 16.2.12; React 19.2.4; TypeScript 5; ECharts 6.1
Other libraries	Commons CSV 1.12.0; OpenPDF 2.0.3; Spring Mail
Authentication	Spring Security; opaque session token (24 h); BCrypt (default work factor); AWS connection token (30 min)
Run configuration	Local: backend port 8080, frontend localhost:3000, PostgreSQL localhost:5432
Deployment	Local run only; no Docker or cloud scripts; HTTPS not configured
Source code	github.com/Mayurj5842/ CloudCostOptimizaionPlatform, main branch, 20 commits
Dataset	One AWS account; billing month August 2026 (Table V)
Analytics	Table II (threshold 50%, minimum 8 daily points)
Tests	9 backend test classes (pass/fail [NOT MEASURED]); no frontend tests
Tools	Postman, Git and GitHub
Source code	github.com/Mayurj5842/ CloudCostOptimizaionPlatform (20 commits)

VII. DATASET AND EVALUATION METHODOLOGY

A. Dataset

The dataset comes from one real AWS account (region ap-south-1 for the resource inventory) and covers the billing month of August 2026 (1 to 31 August). It contains service-level billing records obtained from AWS and stored in PostgreSQL, a billing-summary record, the resource inventory, and the analytical outputs generated by the platform. Table V lists the record counts observed in the stored data. The observation window is a single month, so the dataset is small and suited to functional evaluation rather than statistical benchmarking.

TABLE V
DATASET COMPOSITION

Data Category	Description	Records
AWS Resource Data	Live inventory, ap-south-1 (10 Aug 2026): 1 VPC and 1 security group; no EC2, EBS, S3, RDS, or Lambda resources	2
Billing Data	Service-level rows in the billing table for Aug 2026 (snapshot of 13 Aug 2026)	14
Historical Cost Data	Billing-summary row for the account (total USD 5.38, estimated, synced 13 Aug 2026); month-end total later reported as USD 7.24	1
Other Data	Analytical outputs for Aug 2026: anomalies 4, insights 3, recommendations 5	12 + alerts

B. Collection, Preprocessing, and Evaluation Procedure

Data are collected as described in Section IV, stages 1 to 6. No machine-learning model is used in the analytics reported here: anomaly detection is a threshold rule, and insights and recommendations are deterministic rules (Table II), so there is no training, validation, and test split. The evaluation is functional: the outputs of each module are examined on the real account and cross-checked against the stored billing data. The backend has nine automated test classes covering application startup, billing-source resolution, cost analytics (August 2026 boundaries, previous-period comparison, missing usage dates, no-data handling, and budget calculation), the dashboard, live Cost Explorer fetching and persistence, CUR parsing (manifest, gzip, and malformed files), application-session expiry, notification persistence, and the AWS connection filter; the pass/fail counts from the build are [MVN TEST RESULT]. No automated frontend tests exist, and the API was also exercised manually with Postman. Anomaly precision and recall and recommendation accuracy could not be computed because the account has no labeled anomalies and only one billing month.

The metrics reported are: total and per-service cost, cost share of each service, the number and magnitude of flagged anomalies, the number of insights and recommendations, budget utilization, and API response time (Table VII, to be measured). All values are outputs of the platform as observed in the dashboard.

VIII. RESULTS AND DISCUSSION

All values in this section are actual outputs of the platform for the August 2026 billing month, read from the dashboard (screenshots dated 24 September 2026, unless stated). None is an expected or projected value. The platform computes them with the rules in Table II.

A. AWS Data Retrieval and Cost Visualization

The platform connected to the AWS account, imported billing data into PostgreSQL, fetched live cost from Cost Explorer, and discovered resources. The Overview page (Fig. 3) shows the live AWS cost of USD 7.24 for 1 to 31 August 2026 with service and region breakdowns; the sources were CUR 2.0 and Cost Explorer.

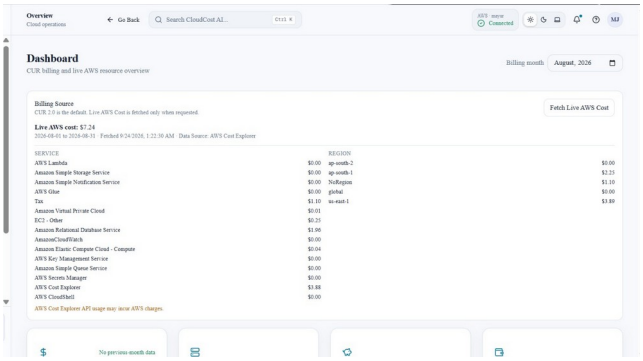


Fig. 3. CloudCost AI overview dashboard showing live AWS cost by service and region.

B. Historical Cost Analysis and Trends

The total for August 2026 was USD 7.24 with a daily average of USD 0.28, as computed by the platform (Fig. 4). The largest cost drivers were AWS Cost Explorer (USD 3.88, 53.58%), Amazon RDS (USD 1.96, 27.07%), Tax (USD 1.10, 15.19%), EC2-Other (USD 0.25, 3.45%), and EC2 compute (USD 0.04, 0.55%). By region, us-east-1 cost USD 3.89, ap-south-1 USD 2.25, and charges without a region USD 1.10. The previous-period comparison was unavailable because only one month of data existed. A notable finding is that the largest single item is the Cost Explorer API itself, showing that monitoring has its own cost

and should be used sparingly; hence the on-demand live fetch.

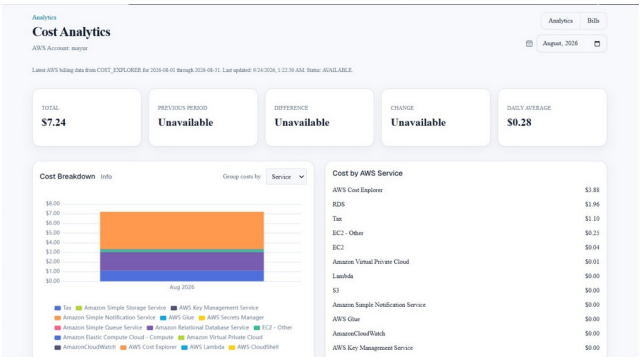


Fig. 4. Cost Analytics page for August 2026: totals, breakdown by service, and cost by AWS service.

C. Forecasting Status

Forecasting is not implemented in the final build. The analytics response model reserves a forecast field, but the platform leaves it empty, and the Forecast page is disabled in the navigation. No forecast results or forecast-error metrics are therefore reported, and forecasting is listed as future work.

D. Anomaly Detection, Insights, and Recommendations

Four days were flagged as anomalies, at 958.62%, 365.52%, 324.14%, and 279.31% above the baseline (Fig. 5), using the rule in (4) and (5). Because the underlying service costs are a few cents, small absolute changes produce large percentages, which shows that percentage rules are sensitive at low baselines. [CONFIRM: the dashboard shows the same date for all four flags; the audited code assigns the first date of the series to every flag. Fix this defect and regenerate Fig. 5 before submission, then state the true dates here.] No ground truth exists, so detection performance is not claimed. AI Insights produced three findings (Fig. 6a): the top cost drivers AWS Cost Explorer (High, 53.58% of spend), Amazon RDS (Medium), and Tax (Medium), with estimated impacts of USD 0.39 and USD 0.20 for the first two, which the platform computes as 10% of service cost and which are therefore heuristic estimates and not verified savings. The Recommendations page produced five findings, four of Medium priority, each with evidence, a suggested action, and Apply and Dismiss controls; potential savings were not available (Fig. 6b). Some recommendations concerned services whose cost was USD 0.00 or a few cents, which suggests that the service-name rule should also require a minimum cost.

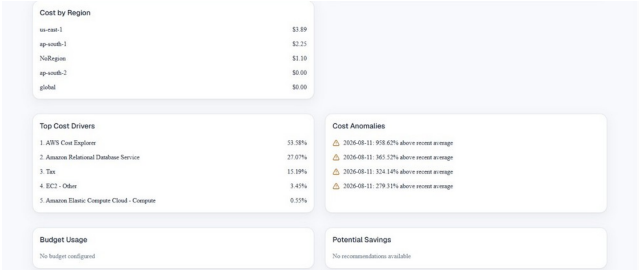


Fig. 5. Cost by region, top cost drivers, and detected cost anomalies for August 2026.

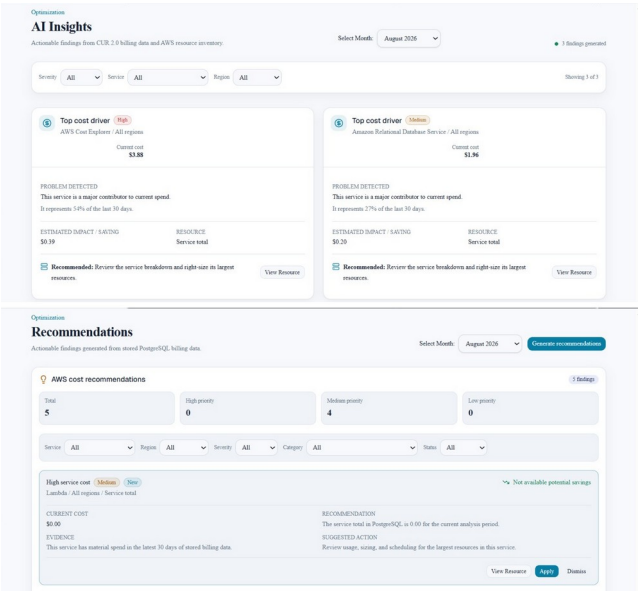


Fig. 6. AI Insights (a) and Recommendations (b) pages.

E. Budgets, Alerts, Reports, Performance, and Security

A monthly budget of USD 10 named "aws production" reached 72.4% utilization (USD 7.24 spent, USD 2.76 remaining, status NORMAL). Alerts were raised at the 50% and 100% thresholds (Fig. 7); the 100% alerts arose from a test budget of USD 2. A background worker evaluates notification preferences every 15 minutes and can deliver in-app and e-mail notifications [CONFIRM: whether e-mail delivery was tested]. A PDF FinOps report with service breakdown, top cost drivers, cost trend, and resource summary can be generated. Authentication used application sessions and the AWS connection token as described in Section VI-E. Numerical API and dashboard response times were [NOT YET MEASURED: fill in Table VII]. Because there is no baseline experiment, Table VI lists the observed outputs rather than a baseline comparison.

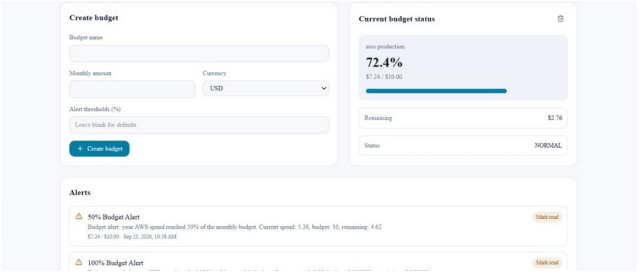


Fig. 7. Budgets & Alerts page showing budget utilization and threshold alerts.

TABLE VI
OBSERVED OUTPUTS, AUGUST 2026 (SINGLE AWS ACCOUNT)

Output	Value	Type
Total cost	USD 7.24	Actual
Daily average	USD 0.28	Actual
Top driver	Cost Explorer, 53.58%	Actual
Second driver	RDS, 27.07%	Actual
Anomalies flagged	4 days	Actual
AI insights	3	Actual
Recommendations	5 (4 Medium)	Actual
Budget utilization	72.4% of USD 10	Actual

Output	Value	Type
Forecast	Not implemented	-

TABLE VII
API RESPONSE TIME (TO BE MEASURED)

Endpoint	Median (ms)	95th pct (ms)
Login	[VALUE]	[VALUE]
Cost analytics	[VALUE]	[VALUE]
Recommendations	[VALUE]	[VALUE]
Report (PDF)	[VALUE]	[VALUE]

IX. ABLATION STUDY

CloudCost AI comprises several analytical modules: cost analysis, anomaly detection, insights, recommendations, and budgets and alerts. **An ablation study was not conducted as part of the current implementation.** A module-wise ablation is listed as future work in Section XI.

X. SECURITY, PRIVACY, SCALABILITY, AND LIMITATIONS

A. Security

Passwords are stored as salted BCrypt hashes, which resist offline guessing [13]. Application sessions are random 32-character tokens with a 24-hour expiry, stored server-side and deleted at logout, and AWS data requests additionally need a 30-minute connection token. AWS keys supplied by the user are validated with STS, kept only in server memory, and never written to the database. Input validation covers registration, AWS connection, and budget requests. Several weaknesses remain. Most API routes are permitted at the Spring Security level and rely on the custom AWS-connection filter; data are scoped by the connected AWS account, and no check ties an application user to an AWS account; CSRF protection is disabled; and the audited configuration has no rate limiting, no login-attempt limit, no security-header configuration, and no HTTPS enforcement. Deployments beyond local use must add TLS and per-user authorization.

B. Privacy

Billing and resource data can reveal an organization's architecture and activity and are therefore sensitive. The platform displays the AWS account identifier in masked form in its settings page. Responsible handling includes least-privilege, read-only AWS access, restricting each user to their own data, and deleting data on request. AWS credentials are not persisted, and AWS data are scoped to the connected account.

C. Scalability

Because application sessions are stored in PostgreSQL, the backend can in principle be replicated behind a load balancer; however, AWS connection sessions are held in the memory of a single server process, so replication would require sticky routing or a shared session store. PostgreSQL can be scaled through indexing, partitioning of time-series tables, and read replicas. AWS integration is bounded by API quotas and per-request charges, so importing CUR data into PostgreSQL and limiting live Cost Explorer calls are important. These are design considerations; scalability testing was not performed.

D. Limitations

(1) The evaluation uses one small AWS account (USD 7.24 for the month, few resources) and one month of history, so results demonstrate functionality rather than performance at scale. (2) With no previous period, trend comparison could not be shown. (3) Forecasting is not implemented. (4) Anomaly detection is a fixed 50% rule against the mean of the earlier days, needs at least eight daily points, is sensitive at low baselines, has no severity, and has no labeled ground truth; the current implementation also attributes flags to the first date of the series [CONFIRM: remove once fixed]. (5) Insights and recommendations are deterministic rules, their estimated impact is a fixed 10% of cost, and savings are mostly unavailable. (6) Billing synchronization is manual, weekly aggregation is absent, and the AWS Budgets API is not called. (7) The implementation targets AWS only, depends on AWS APIs and quotas, and incurs Cost Explorer charges. (8) Response-time measurements and scalability tests have not been performed. (9) The comparison with existing tools is qualitative, and no statistical testing was done. (10) Access control lacks a user-to-account authorization check, rate limiting, and HTTPS enforcement (Section X-A). (11) Pass/fail results of the backend test suite and runtime measurements were not recorded, and there are no frontend tests. (12) The system was run locally without Docker or cloud deployment. (11) The AWS data endpoints are scoped by the AWS connection token, and no check ties an application user to an AWS account. (12) There is no rate limiting or login-attempt limit, no global exception handler, no frontend automated tests, and backend test results were [NOT RECORDED].

XI. FUTURE SCOPE

The following enhancements are proposed and are not part of the current implementation:

- Cost forecasting, starting with statistical time-series methods [14] and then machine-learning models, building on approaches such as those in [7], [8], validated on longer histories.
- Multi-cloud support, with integration of Azure and Google Cloud.
- Automated rightsizing recommendations and, subsequently, automated optimization actions with human approval.
- More sophisticated anomaly detection, including learned detectors [12], with correct date attribution, severity levels, and evaluation against labeled cost events.
- Scheduled automatic billing import and additional notification channels, for example messaging platforms.
- More advanced FinOps analytics, including allocation and unit economics.
- Explainable recommendations that state the evidence for each suggestion.

- Stronger access control: per-user ownership of AWS accounts, role-based authorization, rate limiting, HTTPS, and a global exception handler.
- Integration with additional cloud-management tools.
- Large-scale evaluation across multiple AWS environments and longer histories, with statistical validation, load testing, and a module-wise ablation study.

XII. CONCLUSION

This paper presented CloudCost AI, a platform that validates AWS credentials, imports billing data and discovers resources, stores them in PostgreSQL, and provides cost analysis, top-cost-driver insights, threshold-based anomaly flags, rule-based recommendations, budget tracking with in-app and e-mail alerts, and PDF reports through a session-protected dashboard built on Spring Boot and Next.js. It addresses the fragmentation of cost monitoring, analysis, and advice by providing a unified approach for AWS cost monitoring and analysis. On one AWS account for August 2026, it reported a total cost of USD 7.24, identified AWS Cost Explorer and Amazon RDS as the leading cost drivers, flagged four anomalies, and generated five recommendations. The evaluation is functional and small in scale, the analytics are deterministic rules, forecasting is not yet implemented, and comparison with existing tools is qualitative. Future work will add validated forecasting, learned anomaly detection, multi-cloud support, and evaluation at larger scale.

ACKNOWLEDGMENT

The authors thank Prof. Sarin Deore, project guide, for guidance during this project.

REFERENCES

[1] P. Mell and T. Grance, “The NIST definition of cloud computing,” National Institute of Standards and Technology, Gaithersburg, MD, USA, Spec. Publ. 800-145, Sep. 2011.

[2] M. Armbrust *et al.*, “A view of cloud computing,” *Commun. ACM*, vol. 53, no. 4, pp. 50–58, Apr. 2010.

[3] J. Storment and M. Fuller, *Cloud FinOps*, 2nd ed. Sebastopol, CA, USA: O’Reilly Media, 2023.

[4] FinOps Foundation, “Managing cloud cost anomalies,” FinOps Foundation Working Group. [Online]. Available: https://www.finops.org/wg/managing-cloud-cost-anomalies/

[5] Amazon Web Services, “AWS Cost Explorer Service API reference.” [Online]. Available: https://docs.aws.amazon.com/aws-cost-management/latest/APIReference/

[6] S. Deochake, “ABACUS: A FinOps service for cloud cost optimization,” arXiv:2501.14753, 2024.

[7] P. Nawrocki and M. Smendowski, “FinOps-driven optimization of cloud resource usage for high-performance computing using machine learning,” *J. Comput. Sci.*, vol. 79, Art. no. 102292, 2024, doi: 10.1016/j.jocs.2024.102292.

[8] M. Smendowski and P. Nawrocki, “Optimizing multi-time series forecasting for enhanced cloud resource utilization based on machine learning,” *Knowl.-Based Syst.*, vol. 304, Art. no. 112489, 2024, doi: 10.1016/j.knosys.2024.112489.

[9] N. P. A. Vo, M. Kesarwani, R. Mahindru, and C. Narayanaswami, “FinOps agent—A use-case for IT infrastructure and cost optimization,” arXiv:2510.25914, Oct. 2025.

[10] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” *ACM Comput. Surv.*, vol. 41, no. 3, pp. 1–58, 2009.

[11] P. Nawrocki and M. Smendowski, “A survey of cloud resource consumption optimization methods,” *J. Grid Comput.*, vol. 23, no. 1, Art. no. 5, 2025, doi: 10.1007/s10723-024-09792-0.

[12] M. S. Islam, M. S. Rakha, W. Pourmajidi, J. Sivaloganathan, J. Steinbacher, and A. Miranskyy, “Anomaly detection in large-scale cloud systems: An industry case and dataset,” in *Proc. IEEE/ACM 47th Int. Conf. Softw. Eng.: Softw. Eng. Pract. (ICSE-SEIP)*, 2025; arXiv:2411.09047.

[13] N. Provos and D. Mazières, “A future-adaptable password scheme,” in *Proc. USENIX Annu. Tech. Conf.*, Monterey, CA, USA, 1999, pp. 81–91.

[14] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: https://otexts.com/fpp3/